C. Wet Shark and Flowers

time limit per test: 2 seconds
memory limit per test: 256 megabytes

There are n sharks who grow flowers for Wet Shark. They are all sitting around the table, such that sharks i and $i + 1$ are neighbours for all i from 1 to $n - 1$. Sharks n and 1 are neighbours too.

Each shark will grow some number of flowers s_i. For i-th shark value s_i is random integer equiprobably chosen in range from l_i to r_i. Wet Shark has it's favourite prime number p, and he really likes it! If for any pair of **neighbouring** sharks i and j the product $s_i \cdot s_j$ is divisible by p, then Wet Shark becomes happy and gives 1000 dollars to each of these sharks.

At the end of the day sharks sum all the money Wet Shark granted to them. Find the expectation of this value.

Input

The first line of the input contains two space-separated integers n and p ($3 \le n \le 100\,000, 2 \le p \le 10^9$) — the number of sharks and Wet Shark's favourite prime number. It is guaranteed that p is prime.

The i-th of the following n lines contains information about i-th shark — two space-separated integers l_i and r_i ($1 \le l_i \le r_i \le 10^9$), the range of flowers shark i can produce. Remember that s_i is chosen equiprobably among all integers from l_i to r_i, inclusive.

Output

Print a single real number — the expected number of dollars that the sharks receive in total. You answer will be considered correct if its absolute or relative error does not exceed 10^{-6}.

Namely: let's assume that your answer is a, and the answer of the jury is b. The checker program will consider your answer correct, if $\frac{|a-b|}{\max(1,b)} \le 10^{-6}$.

Examples

input Copy

```
3 2
1 2
420 421
420420 420421
```

output Copy

```
4500.0
```

input Copy

```
3 5
1 4
2 3
11 14
```

output Copy

```
0.0
```

- There are n sharks sitting in a circle
- Shark i chooses a random integer s_i uniformly from [l_i, r_i]
- Choices of all sharks are independent
- Wet Shark likes a prime p

For each neighbouring pair (i, j):

- If s_i * s_j is divisible by p
- Then each of the two sharks gets 1000 dollars

We want the expected total money paid.

Define the random variable

Let:

X = total money paid

We want:

E[X]

By linearity of expectation:

E[X] = sum of expected money from each neighbouring pair

This works even though events overlap (a shark can be in two pairs).

Focus on ONE neighbouring pair

Take a fixed neighbouring pair (i, j).

Define a random variable:

X_{i,j} = money given because of this pair

What values can X_{i,j} take?

- If s_i * s_j divisible by p → both get 1000
  → total = 2000

- Otherwise → 0

So:

E[X_{i,j}] = 2000 × P(p divides s_i * s_j)

That's it.

When is p | (s_i * s_j) ?

Because p is prime, this is crucial:

p divides s_i * s_j
iff p divides s_i or p divides s_j

Probability that p divides a random number in a range

For shark i:

- s_i is uniform in [l_i, r_i]
- Count how many numbers in this range are divisible by p

Number of multiples of p in [l_i, r_i]:

cnt_i = floor(r_i / p) - floor((l_i - 1) / p)

Total numbers:

len_i = r_i - l_i + 1

So:

P(p divides s_i) = cnt_i / len_i

Let:

q_i = P(p divides s_i)

Compute P(p divides s_i * s_j)

We want:

P(p | s_i OR p | s_j)

Use inclusion–exclusion:

P = q_i + q_j - q_i * q_j

Why?

- q_i = probability p | s_i
- q_j = probability p | s_j
- Subtract overlap (both divisible)

This uses independence of s_i and s_j.

Expected contribution of one pair

For neighbours (i, j):

E[X_{i,j}] = 2000 × (q_i + q_j - q_i * q_j)

Algorithm

1. Read n, p
2. For each shark i:
   - Compute q_i = P(p divides s_i)
3. For each neighbour (i, i+1) and (n,1):
   - Add q_i + q_j - q_i*q_j to sum
4. Multiply final sum by 2000
5. Print answer

Code

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long p;
    cin >> n >> p;

    vector<double> q(n);
    for (int i = 0; i < n; i++) {
        long long l, r;
        cin >> l >> r;

        long long cnt = r / p - (l - 1) / p;
        long long len = r - l + 1;

        q[i] = (double)cnt / (double)len;
    }

    double expected = 0.0;
    for (int i = 0; i < n; i++) {
        int j = (i + 1) % n;
        expected += q[i] + q[j] - q[i] * q[j];
    }

    expected *= 2000.0;
    cout << fixed << setprecision(10) << expected << "\n";

    return 0;
}
```